Robust Training of Neural Networks at Arbitrary Precision and Sparsity

C. Ye, G. Chu, Y. Liu, Y. Zhang, L. Lew, L. Zhang, M. Sandler, A. Howard
Google DeepMind • ICLR 2026

Presented by

Ayush Srivastava (2026AIB2015) Sarvesh Nikas (2026AIB2045)

Why quantization, and the forward pass during training

Why quantization.

- Billions of weights, each 32 or 16 bits.
- Too big for a phone, slow, costly in energy.
- Quantization: 4, 2 or 1 bit per weight.

Training with quantization.

- Training keeps the float weights x.
- Forward pass uses quantized $\hat{x}$; backward pass updates the float x.
- Inference keeps only $\hat{x}$ (stored as q plus s_f, b_f per block).

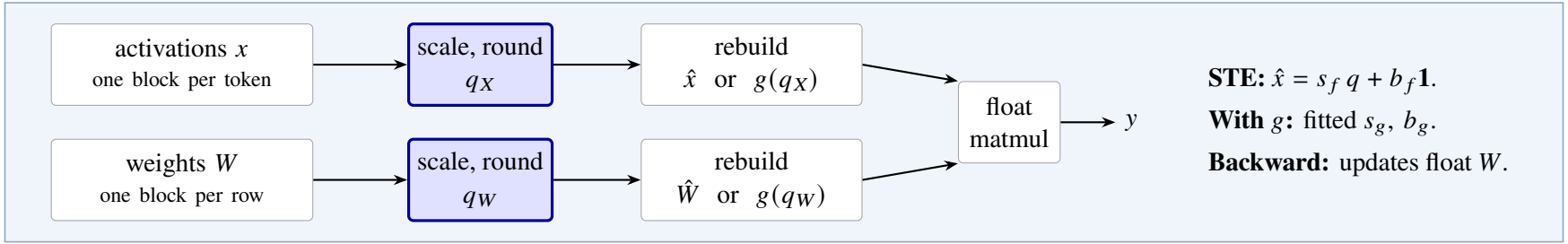
The forward pass during training.

$$x \xrightarrow{f} f(x) \xrightarrow{\text{round}} q \xrightarrow{s_f q + b_f \mathbf{1}} \hat{x} \longrightarrow \text{rest of network} \longrightarrow L$$

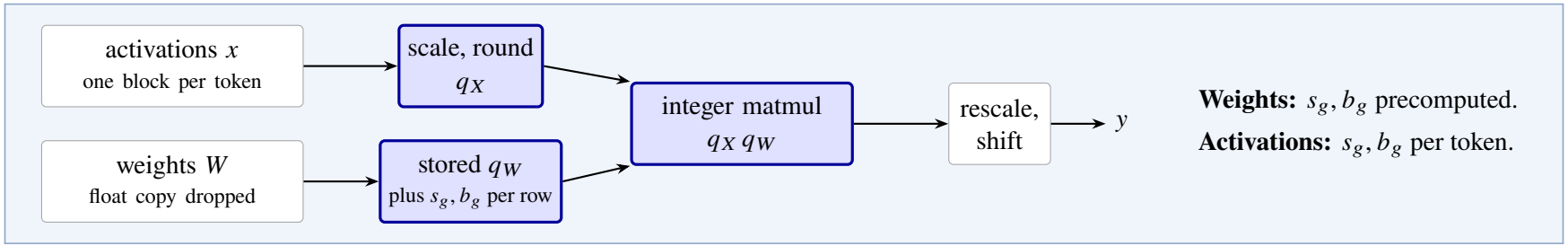
- $x \in \mathbb{R}^N$: float block, kept and updated during training.
- $f(x) = \dfrac{x - b_f \mathbf{1}}{s_f}$: rescales x onto the k-bit integer grid $\{0, \ldots, 2^k - 1\}$, with $s_f = \dfrac{x_{\max} - x_{\min}}{2^k - 1}$, $b_f = x_{\min}$.
- $q = \text{round}\big(f(x)\big) = f(x) + \delta$: the integers, with rounding error $|\delta_i| \le \frac{1}{2}$.
- $\hat{x} = s_f\, q + b_f \mathbf{1} = x + s_f\, \delta$: x as seen after quantization; this is what the next layer uses.

What gets quantized, and where

Training time. Rebuild in float, so gradients can flow.



Inference time. Multiply integers only; apply scales afterwards.



The problem: the true gradient is zero

Chain rule through the quantizer. Gradient descent needs $\nabla_x L$:

$$\nabla_x L = \left(\frac{\partial \hat{x}}{\partial x}\right)^\top \nabla_{\hat{x}} L, \qquad \frac{\partial \hat{x}}{\partial x} = \underbrace{s_f}_{\text{scale back}} \cdot \underbrace{\frac{\partial q}{\partial f(x)}}_{\text{round}} \cdot \underbrace{\frac{1}{s_f}}_{f} = \frac{\partial q}{\partial f(x)}.$$

The two scalings cancel: the local gradient is exactly the slope of round.

Round is a staircase.

$$\frac{d}{du}\,\text{round}(u) = 0 \quad \text{for } u \notin \mathbb{Z} + \tfrac{1}{2}, \qquad \text{undefined at } u \in \mathbb{Z} + \tfrac{1}{2}.$$

Hence $\dfrac{\partial q}{\partial f(x)} = 0$ almost everywhere, so

$$\nabla_x L = 0.$$

Consequence. A small change in x does not change q, so the loss gives x no direction to move: x and every layer before it never update. Training cannot start.

The standard fix: Straight-Through Estimator (STE)

Idea. Round as before in the forward pass, but treat the rounding error as a constant in the backward pass:

$$q = f(x) + \delta, \qquad \delta = \text{stopgrad}\Big(\text{round}\big(f(x)\big) - f(x)\Big),$$

where stopgrad keeps the value in the forward pass and has zero derivative in the backward pass.

Forward vs. backward.

$$\text{forward: } \hat{x} = x + s_f\,\delta, \qquad \text{backward: } \frac{\partial \hat{x}}{\partial x} = s_f \cdot I \cdot \frac{1}{s_f} = I, \quad \nabla_x L = \nabla_{\hat{x}} L.$$

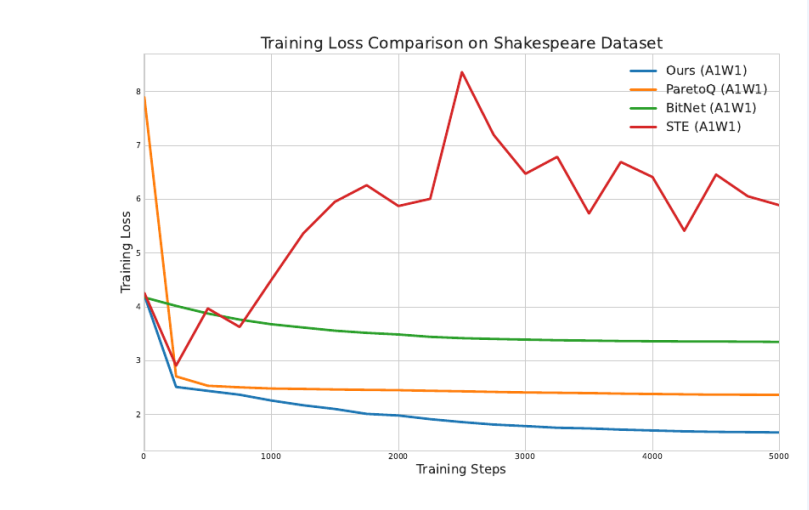
The gradient reaching x is no longer zero, so training works.

The blind spot.

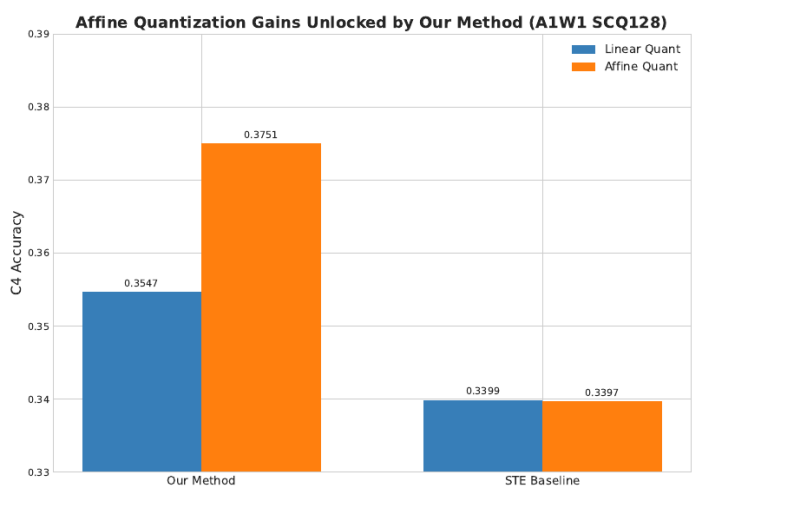
- The forward pass shifts every entry by $s_f \delta_i$, but the Jacobian is I, exactly as with no quantization. δ does not appear in it.
- So earlier layers get no signal about how rounding hurts the loss, and cannot learn to be robust to it.
- At 8 bits $s_f \delta$ is tiny and this is harmless. At 1–2 bits it is as large as the values themselves: STE diverges.

Reference: Y. Bengio, N. Léonard, A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," arXiv:1308.3432, 2013.

STE at 1 bit: results from the paper



Training loss, 1-bit weights and activations (A1W1), small GPT on Shakespeare. STE (red) diverges; the paper's method (blue) converges.



Accuracy at A1W1, linear vs. affine quantization. Under STE affine gives no gain; with the paper's method it does.

Source: Ye et al., "Robust Training of Neural Networks at Arbitrary Precision and Sparsity," ICLR 2026, Fig. 1.

The proposed approach: a denoising reconstruction g

The change. Keep f and round. Replace $\hat{x}$ by a fitted reconstruction g.

Reconstruction.
A new affine map.

$$\hat{x} = s_f\, q + b_f \mathbf{1} \quad \downarrow \quad g(q) = s_g\, q + b_g \mathbf{1}$$

$\Longrightarrow$

Ridge regression objective.
Fit s_g and b_g to x.

$$\min_{s_g, b_g} \frac{1}{2N} \left\| s_g\, q + b_g \mathbf{1} - x \right\|^2 + \frac{\lambda}{2} s_g^2$$

$\Longrightarrow$

Solution.
Ridge regression has a closed form.

$$s_g = \frac{\mathrm{Cov}_{xq}}{\mathrm{Var}_q + \lambda}, \qquad b_g = \bar{x} - s_g\, \bar{q}$$

λ is the regularization parameter.

Training with g. One forward and one backward pass.

Forward pass. The next layer gets $g(q)$, not $\hat{x}$.

$$x \xrightarrow{f} f(x) \xrightarrow{\text{round}} q \xrightarrow{g} g(q) \longrightarrow \text{rest of network} \longrightarrow L$$

Backward pass. Round stays straight through, as in STE. But g now depends on δ.

$$\nabla_x L \xleftarrow[\text{straight through}]{\times\, 1/s_f} \nabla_q L \xleftarrow[\text{depends on } \delta]{\times\, (\partial g/\partial q)^\top} \nabla_g L \longleftarrow \text{rest of network} \longleftarrow L$$

Why it works: the backward pass sees δ

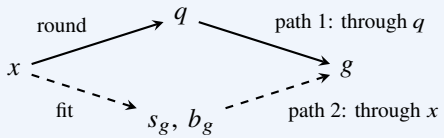
The idea. Both methods replace round's slope by a constant surrogate, so $\partial q/\partial x = I/s_f$. Only the step after round differs.

STE. $\hat{x} = s_f\, q + b_f \mathbf{1}$ has the same slope for every δ.

$$\nabla_x L = \underbrace{\left(\frac{1}{s_f}\,\frac{\partial \hat{x}}{\partial q}\right)^{\top}}_{\text{local} = I}\ \underbrace{\nabla_{\hat{x}} L}_{\text{upstream}} = \nabla_{\hat{x}} L$$

With g. The gradient reaches x by two paths. Local gradient times upstream gradient:

$$\nabla_x L = \underbrace{\left(\underbrace{\frac{1}{s_f}\,\frac{\partial g}{\partial q}}_{\text{through } q} + \underbrace{\frac{\partial g}{\partial x}}_{\text{through } x}\right)^{\top}}_{\text{local}}\ \underbrace{\nabla_g L}_{\text{upstream}}$$



Through q. Changes with the rounding error. It carries δ into the gradient:

$$\frac{\partial g}{\partial q} = \underbrace{s_g\, I}_{\substack{\text{like STE's } s_f I,\\ \text{but } s_g \text{ carries } \delta}} + \underbrace{q\left(\frac{\partial s_g}{\partial q}\right)^{\top} + \mathbf{1}\left(\frac{\partial b_g}{\partial q}\right)^{\top}}_{\substack{\text{new terms,}\\ \text{carry } \delta \text{ through } x - g(q)}}$$

Through x. For the current q, g is linear in x.

No information about the rounding error.

Results from the paper

Gemma 3 language model (1B), trained from scratch. Score: share of next tokens predicted correctly.

bits for weights and activations	STE	with g	gain
4	0.425	0.427	+0.002
2	0.379	0.403	+0.023
1	0.340	0.375	+0.035

The fewer the bits, the bigger the gain.

Extension to sparsity.
Pruned and quantized 4B beats full 1B.

- 4B, 1-bit weights, half pruned: 650 MB, score 0.452.
- 1B, full precision: 1400 MB, score 0.449.

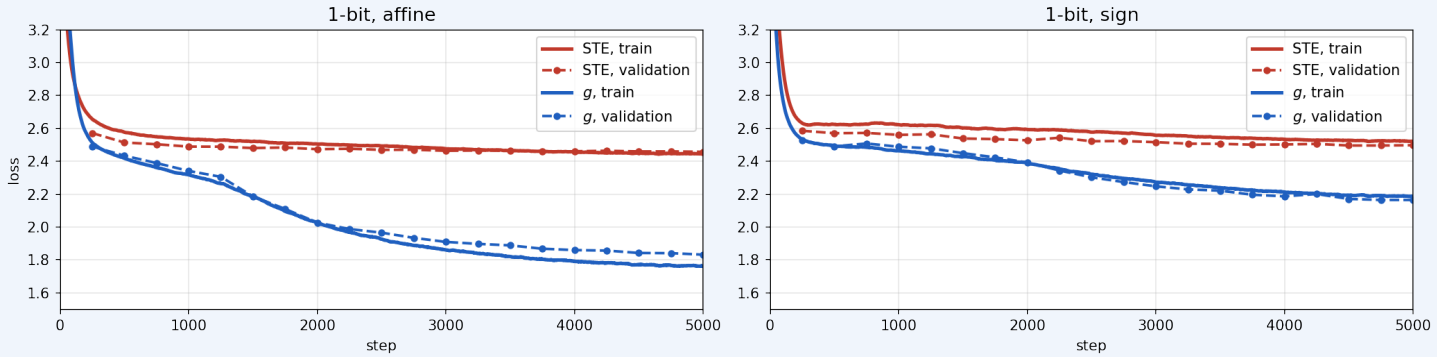
Other domains.
At 4 bits, images and translation match full precision.

- Images (ResNet-50): 76.45% at 4 bits, 76.41% full precision.
- Translation: 29.71 BLEU at 4 bits, 29.49 full precision.

Our implementation 1: the paper's 1-bit GPT

Setup. Small GPT on Shakespeare, the paper's exact settings, 1-bit weights and activations.

Paper's exact setting, 5000 steps: STE stalls, g keeps learning



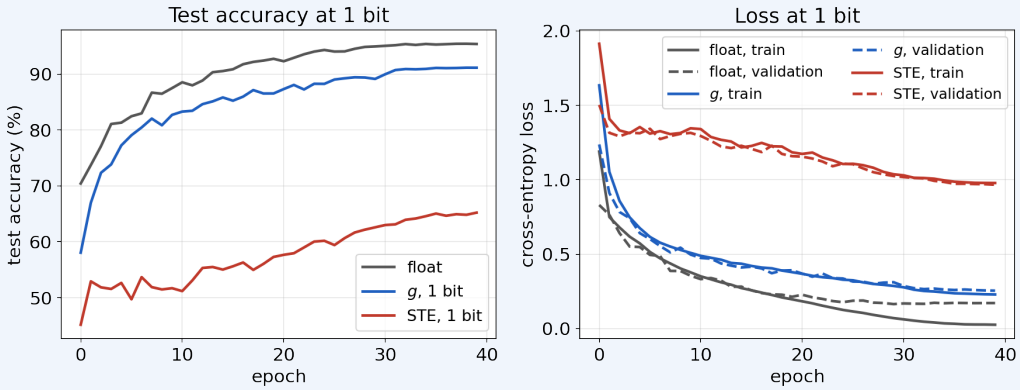
- **STE stalls, g keeps learning.** Final loss 2.41 against 1.65.
- **g reproduces the paper's curve.** Within 0.05 at every checkpoint.
- **STE does not blow up here.** The paper's STE climbs to about 6.

Reference: model and recipe follow nanoGPT by A. Karpathy, `github.com/karpathy/nanoGPT`.

Our implementation 2: solving a real-world task

Task. Label each satellite image as one of 10 land types, such as forest or river.

Model. A small vision transformer, quantized to different precisions.



Test accuracy (%).

	4 bits	2 bits	1 bit
STE	95.4	93.3	65.2
with g	95.6	94.5	**91.2**
gain	**+0.3**	**+1.3**	**+25.9**

Full precision: 95.4%.

The gain over STE grows as bits drop.

Data: P. Helber et al., "EuroSAT: a novel dataset and deep learning benchmark for land use and land cover classification," IEEE JSTARS, 2019.

Advantages and disadvantages

Advantages.

- **Makes 1-bit training work.**

  The fitted reconstruction keeps training stable.

- **Better reconstruction.**

  Closer to x than standard dequantization.

- **Extends to sparsity.**

  Pruning is one more rounding step, then g:

  $$x \xrightarrow{\text{threshold}} x_S \xrightarrow{f,\ \text{round}} q \xrightarrow{g} g(q)$$

Disadvantages.

- **Round still uses a surrogate.**

  "Without gradient estimation" is too strong.

- **Small gain at 4 bits.**

  Only +0.002, *in the most common setting*.

- **Savings are estimates.**

  No real hardware, no speed numbers.

Our takeaways

What convinced us.

- **A clean comparison.**

  Only the reconstruction differs between the two methods.

- **Consistent gains.**

  g wins in every setting, more at fewer bits.

- **Our 1-bit run agrees.**

  STE stalls; g keeps learning.

Open question.

"No gradient estimation".

The paper claims it (abstract; Table 8).

But δ is detached, exactly as in STE:

$$q = f(x) + \text{stopgrad}(\delta) \;\Rightarrow\; \partial q / \partial f(x) = I$$

Takeaway

A good approximation can still cause optimization issues.

Rounding error looks ignorable, but training needs its information.

Fitting parameters to the data is a good bet.